Solution: Find if Doubly Linked List is a Palindrome (easy)

Problem Statement

Given a doubly linked list, determine whether it is a palindrome.

A doubly linked list is a palindrome if it reads the same backward as forward, utilizing the previous and next pointers of the nodes.

Examples

1. **Example 1**:
   - **Input**: 1 <-> 2 <-> 3 <-> 2 <-> 1
   - **Output**: true
   - **Justification**: The list reads the same backward as forward.
2. **Example 2**:
   - **Input**: 1 <-> 2 <-> 2 <-> 3
   - **Output**: false
   - **Justification**: Reading backward, the list is 3 <-> 2 <-> 2 <-> 1, which is not the same as reading forward.
3. **Example 3**:
   - **Input**: 1 <-> 1 <-> 1 <-> 1
   - **Output**: true
   - **Justification**: All elements are the same, so the list is a palindrome.

Solution

To determine whether a doubly linked list is a palindrome, we can utilize two pointers approach. Initialize two pointers, one at the start and one at the end of the doubly linked list. Compare the elements at these pointers. If the elements are equal, move the start pointer one step forward and the

end pointer one step backward and compare again. If at any point the elements at the pointers are not equal, the list is not a palindrome. Continue this process until the start pointer is greater than or equal to the end pointer. If the loop finishes without finding unequal elements, the list is a palindrome.

Here is the step-by-step solution.

1. **Check for Base Cases**:
   - If the head of the doubly linked list is `null` or if there is only one node (head's next is `null`), return `true`.
2. **Find the Tail Node**:
   - Start with the head node and traverse to the end of the list to find the tail node. This is achieved by iterating through the nodes using `next` pointers until you reach a node where `next` is `null`.
3. **Initialize Two Pointers**:
   - Two pointers are initialized: `start` at the head of the list and `end` at the tail of the list.
4. **Palindrome Check Loop**:
   - The list is traversed from both ends simultaneously towards the middle. In each iteration:
     - Compare the values of the nodes pointed by `start` and `end`.
     - If the values are not equal at any point, return `false`, indicating that the list is not a palindrome.
     - Move `start` forward (to the next node) and `end` backward (to the previous node).
5. **Return Result**:
   - If the loop completes without returning `false`, it means all mirrored elements in the list are equal. Hence, the function returns `true`, confirming that the list is a palindrome.

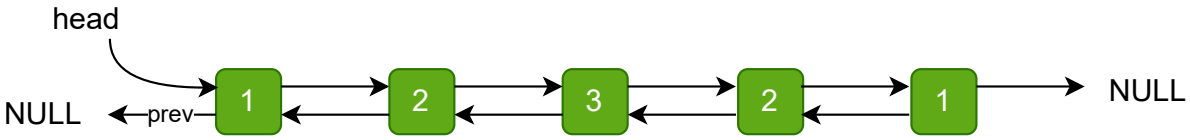
This algorithm is effective because it leverages the nature of a doubly linked list, using both next and previous pointers to compare elements from both ends of the list, reducing the time complexity. As we are not using any additional data structures to store elements of the list and only using constant extra space, it is space-efficient as well.

Algorithm Walkthrough:

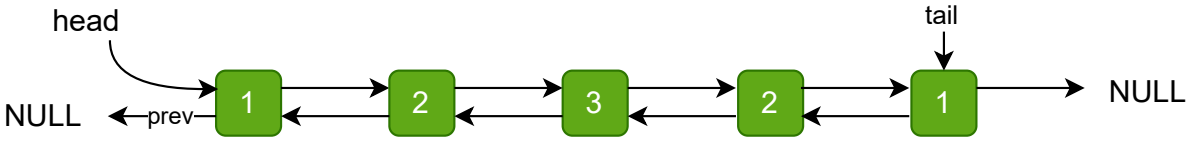
- Initialize two pointers: `start` at the head of the list and `end` at the tail.
- While the `start` pointer is less than the `end` pointer:
  - Compare the values at the `start` and `end` pointers.
  - If they are not equal, return false.
  - Move the `start` pointer one step forward and the `end` pointer one step backward.

- If the loop finishes without returning, return true, as the list is a palindrome.

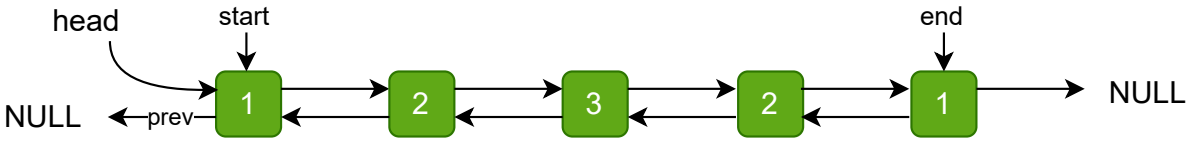
Here is the visual representation of the algorithm:

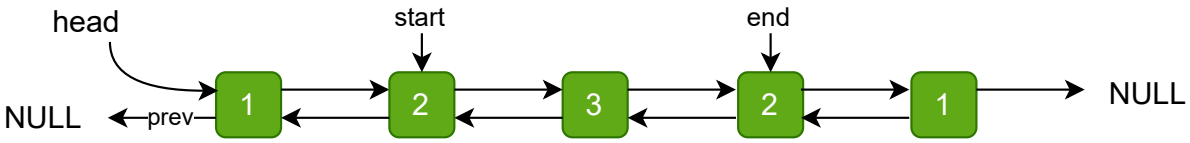


First of all, we will find the tail

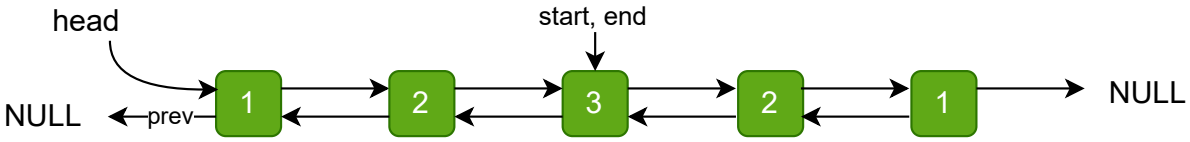


We will create two variables which will point to **start** and **end** respectively and will help in traversal.
Since values are same at start and end, we will move **start** variable forward to point to the **next node** and **end** will be moved in backward position



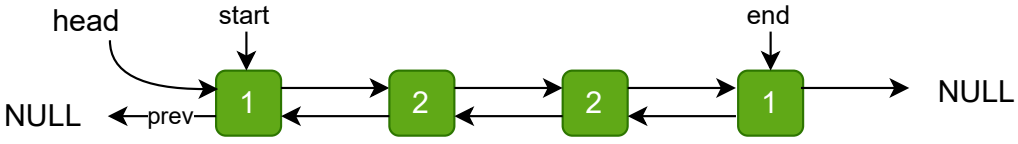


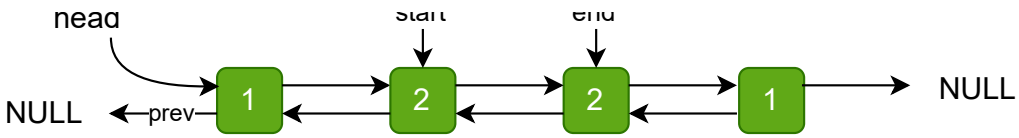
Now, both **start** and **end** are pointing to the same node so loop will be terminated and **true** will be returned



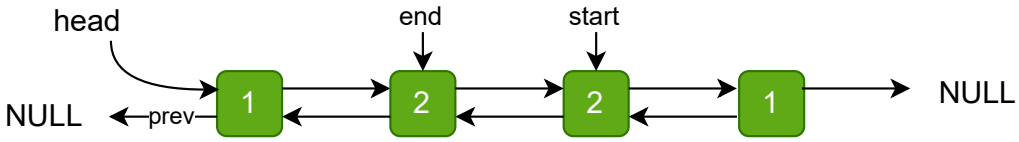
Output: true

Let's take another example and we will first find the tail and then **start** and **end** variables will be initialized:





Now **start.prev** is pointing to the **end** and it means the whole double linkedlist is traversed and we will exit the loop



Output: true

Find if DLL is Palindrome or not

Code

Here is the code for this algorithm:

Java

```java
// class DLNode {
//     int val;
//     DLNode next, prev;
//     DLNode(int val) { this.val = val; }
// }

class Solution {
    public boolean isPalindrome(DLNode head) {
        if (head == null || head.next == null) return true;

        DLNode tail = head;
        while (tail.next != null) tail = tail.next; // Find the tail of the doubly linked list

        DLNode start = head;
        DLNode end = tail;

        while (start != end && start.prev != end) {
            if (start.val != end.val) return false; // Compare the values at start and end pointers
            start = start.next;
            end = end.prev;
        }

        return true;
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The algorithm traverses the doubly linked list twice: once to find the tail of the list and once to compare the elements for palindrome checking. Each traversal takes O(n) time, where n is the number of elements in the list.

1. **Finding the Tail:** We traverse the list from the head to the tail, which takes O(n) time.
2. **Comparing Elements:** We use two pointers, one starting from the head and the other from the tail, moving towards the center of the list. In the worst case, this also takes O(n) time.

Therefore, the total time complexity of the algorithm is O(n) + O(n) = O(2n), which is asymptotically equivalent to O(n).

Space Complexity

The algorithm uses only a constant amount of extra space to store the pointers (start, end, tail) and the input does not affect the space usage. Therefore, the space complexity of the algorithm is O(1), which means it uses constant space.

← Previous

Problem 4: Find If Doubly Linked List Is A Palindrome (Easy)

Next →

Problem 5: Swap Nodes In Pairs (Medium)

Mark as Completed